Description of running the application

Starting Minicube k8s cluster:

```
minikube start
```

Setting Docker Env to use the Minicube Internal Docker daemon in current terminal session:

```
eval $(minikube docker-env)
```

So now:

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % docker images

IMAGE                                              ID             DISK USAGE   CONTENT SIZE   EXTRA
gcr.io/k8s-minikube/storage-provisioner:v5         ba04bb24b957         29MB             0B    U
registry.k8s.io/coredns/coredns:v1.13.1            e08f4d9d2e6e       73.4MB             0B    U
registry.k8s.io/etcd:3.6.6-0                       271e49a0ebc5       59.8MB             0B    U
registry.k8s.io/kube-apiserver:v1.35.1             7459ea001763       83.9MB             0B    U
registry.k8s.io/kube-controller-manager:v1.35.1    464c974e702e       71.1MB             0B    U
registry.k8s.io/kube-proxy:v1.35.1                 2d611d040292       72.5MB             0B    U
registry.k8s.io/kube-scheduler:v1.35.1             4cad4f996631       48.7MB             0B    U
registry.k8s.io/pause:3.10.1                       d7b100cd9a77        514kB             0B    U
```

Install NFS server and provisioner via Helm in the cluster

Updating local list of charts:

```
helm repo add nfs-ganesha-server-and-external-provisioner
https://kubernetes-sigs.github.io/nfs-ganesha-server-and-external-
provisioner/

helm repo update
```

Creating the permanend, dedicated 1GB virtual hard drive for NFS server, `persistence.enabled` option enables config persistence using PVC

```
helm install nfs-server nfs-ganesha-server-and-external-provisioner/nfs-
server-provisioner \
  --set persistence.enabled=true \
  --set persistence.size=1Gi \
  --set storageClass.name=nfs-storage
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % helm install nfs-server nfs-ganesha-server-and-external-provisioner/nfs-server-provisioner \
  --set persistence.enabled=true \
  --set persistence.size=1Gi \
  --set storageClass.name=nfs-storage
NAME: nfs-server
LAST DEPLOYED: Tue Apr 14 07:25:41 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
The NFS Provisioner service has now been installed.

A storage class named 'nfs-storage' has now been created
and is available to provision dynamic volumes.

You can use this storageclass by creating a `PersistentVolumeClaim` with the
correct storageClassName attribute. For example:

    ---
    kind: PersistentVolumeClaim
    apiVersion: v1
    metadata:
      name: test-dynamic-volume-claim
    spec:
      storageClassName: "nfs-storage"
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: 100Mi
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl get storageclass
NAME                 PROVISIONER                                    RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
nfs-storage          cluster.local/nfs-server-nfs-server-provisioner   Delete          Immediate           true                   7m1s
standard (default)   k8s.io/minikube-hostpath                       Delete          Immediate           false                  5d17h
```

Create a Persistent Volume Claim which will bind to a NFS Persistent Volume provisioned dynamically by the provisioner

pvc.yaml

```
kubectl apply -f pvc.yaml
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl apply -f pvc.yaml
persistentvolumeclaim/nfs-pvc created
```

Create a Deployment with a HTTP server (e.g., apache or nginx). The web content directory should be mounted as a volume using the PVC created in the previous step.

The deployment.yaml deploys an nginx web server and ensures the pod is running all the time. It uses PVC to mount the shared volume inside the pod.

```
kubectl apply -f deployment.yaml
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
```

Create a Service associated with the Pod(s) of the HTTP server Deployment.

We create the Service in service.yaml which aim is to expose our pod (the http server) as a network service

```
kubectl apply -f service.yaml
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl apply -f service.yaml
service/nginx-service created
```

Running service:

```
minikube service nginx-service
```

Create a Job which mounts the PVC and copies a sample content through the shared NFS PV.

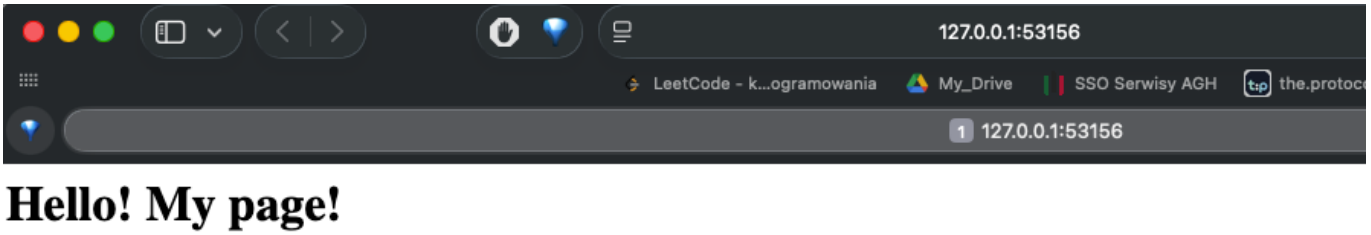
The job.yaml writes the `index.html`, mounts pvc and copies the content to NFS provisioner. Then it terminates.

```
kubectl apply -f job.yaml
```

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl apply -f job.yaml
job.batch/nfs-content-copy-job created
```

Now running service:

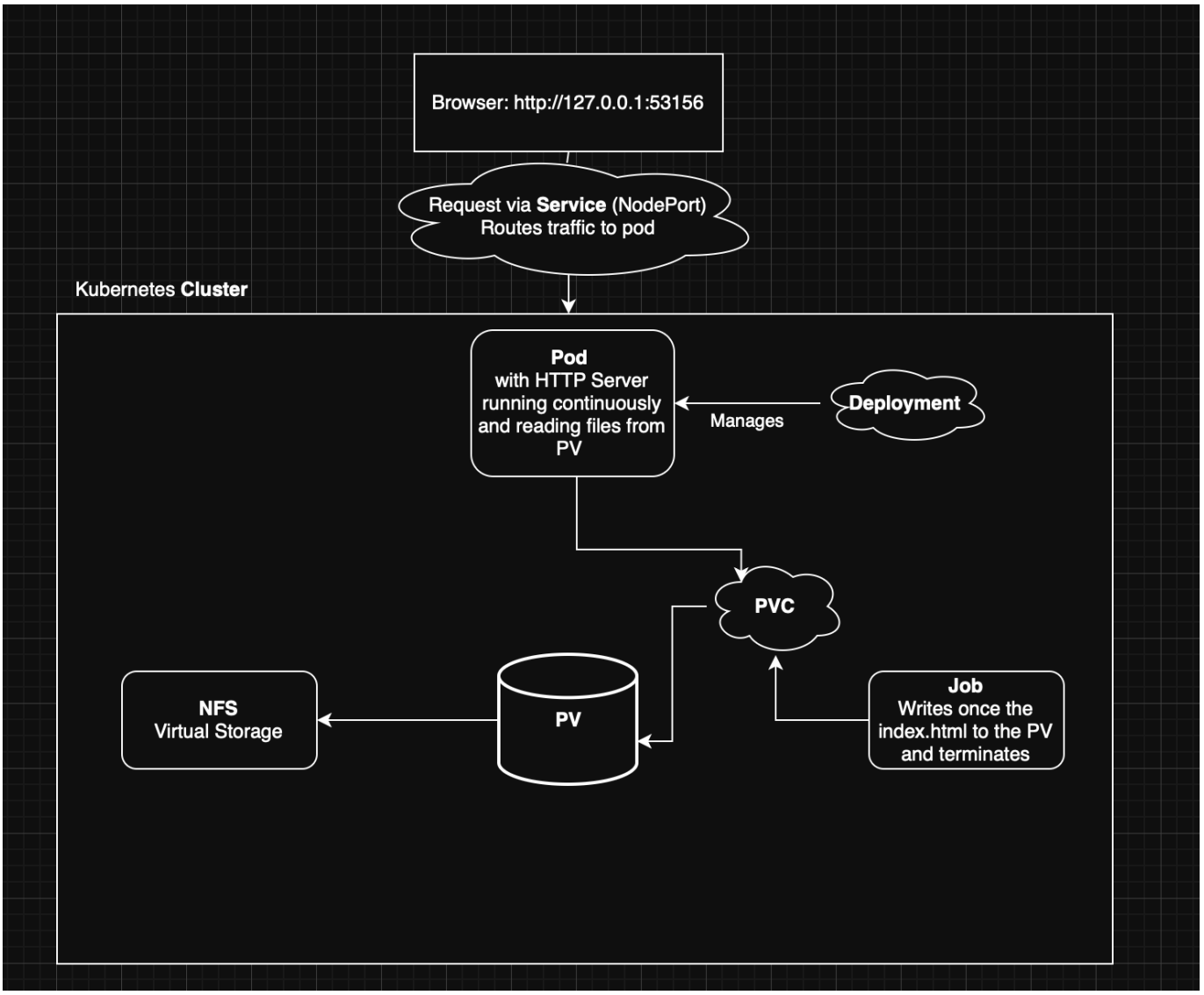
```
minikube service nginx-service
```



The pods status after completing lab:

```
joannaheldak@Joannas-MacBook-Pro lsc-lab6-kubernetes-monkeykingggggg % kubectl get pods
NAME                                    READY   STATUS      RESTARTS   AGE
nfs-content-copy-job-hzr5s              0/1     Completed   0          26m
nfs-server-nfs-server-provisioner-0     1/1     Running     0          98m
nginx-deployment-797b59bc4d-2j96n       1/1     Running     0          61m
```

Architecture diagram with explanation



Browser communication: browser request is routed to NodePort 30 001 on nginx-service. The Service routes the traffic to Nginx Pod on port 80.
Deployment (nginx-deployment): ensures that 1 replica is always running.
Pod: listens to requests from Service. Reads files stores on the PV drive via nfs-pvc.